

22.08 Algoritmos y Estructuras de Datos

Level 1: Warm Up

Week 1

Introducción

En esta primera práctica vamos a simular el sistema solar en 3D.

Physics brushup

La fuerza gravitatoria ejercida sobre el cuerpo de masa m_i ubicado en $\mathbf{x}_i$ por el cuerpo de masa m_j ubicado en $\mathbf{x}_j$ es:

$$\mathbf{F}_{i,j} = - \frac{Gm_i m_j}{\|\mathbf{x}_i - \mathbf{x}_j\|^2} \hat{\mathbf{x}}_{i,j}$$

$\hat{\mathbf{x}}_{i,j}$ es un vector unitario que va del cuerpo j al cuerpo i :

$$\hat{\mathbf{x}}_{i,j} = \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|}$$

La constante gravitatoria G es:

$$G = 6.6743 \times 10^{-11} \frac{N \cdot m^2}{kg^2}$$

La fuerza neta $\mathbf{F}_i$ que actúa sobre el cuerpo i es:

$$\mathbf{F}_i = \sum_{j=1, j \neq i}^N \mathbf{F}_{i,j}$$

Para determinar la aceleración $\mathbf{a}_i$ aplicamos la segunda ley de Newton:

$$\mathbf{a}_i = \frac{\mathbf{F}_i}{m_i}$$

Para calcular la posición $\mathbf{x}_i$ debemos integrar la aceleración dos veces. Pero debemos hacerlo en forma discreta, ya que nuestras computadoras son digitales.

Las siguientes ecuaciones permiten aproximar las integrales de velocidad y de posición con un **time step** Δt :

$$\mathbf{v}_i(n + 1) = \mathbf{v}_i(n) + \mathbf{a}_i(n)\Delta t$$

$$\mathbf{x}_i(n + 1) = \mathbf{x}_i(n) + \mathbf{v}_i(n + 1)\Delta t$$

Si el error de la aproximación es demasiado grande, prueba disminuyendo Δt . Pero cuidado, ¡disminuirlo aumenta también el cálculo!

Nota: Es importante que actualices la posición $\mathbf{x}_i(n)$ con $\mathbf{v}_i(n + 1)$ y no con $\mathbf{v}_i(n)$; hacerlo con $\mathbf{v}_i(n + 1)$ mejora notablemente la precisión.

Organización

Ahora que comprendes el problema, puedes empezar a escribir código. En la primer parte del trabajo vamos a trabajar en **lenguaje C**.

¿Cómo conviene organizar el proyecto? Una buena idea es separarlo en **modelo** y **vista**. El modelo involucra el estado (las variables que describen el estado actual de la simulación) y la actualización (las funciones que actualizan la simulación al próximo estado). La vista involucra la representación gráfica del modelo.

Esta idea se conoce como modelo-vista-controlador. En este proyecto no necesitas controlador ya que el usuario no participa en forma interactiva.

Para implementar la simulación parte del *starter code* que te damos. También debes usar la biblioteca gráfica raylib, que puedes instalar siguiendo los pasos de nuestra guía **CMake y CTest**.

Modelo

Empieza preguntándote qué variables de estado necesitas.

La aceleración de cada cuerpo depende sólo de las fuerzas instantáneas aplicadas sobre él, pero la actualización de velocidad y posición requiere el estado anterior. Por tanto debes almacenar la **posición** y la **velocidad** de cada cuerpo. Para la simulación necesitas además la **masa** del cuerpo, y para la representación gráfica, su **tamaño** y **color**. Puedes encapsular todas estas variables en el tipo de datos `OrbitalBody` del archivo `orbitalSim.h`.

Define asimismo un tipo de datos **OrbitalSim** que contenga el valor del **time step**, el **tiempo transcurrido** desde el comienzo de la simulación, el **número de cuerpos** en la simulación y un **puntero** a un arreglo de **OrbitalBodys**. Esto permite encapsular el estado de la simulación en una sola variable.

Para crear una simulación, completa la función **makeOrbitalSim** que recibe el valor del time step y devuelve un puntero a un **OrbitalSim** inicializado. Aprovecha los datos del archivo **ephemerides.h** que te proveemos en el starter code.

Pasemos a la simulación: completa la función **updateOrbitalSim** para simular un time step. La función recibe como parámetro un puntero a un **OrbitalSim**.

Realiza la simulación en dos pasos: calcula primero todas las aceleraciones, y actualiza recién entonces las posiciones y velocidades. Si no separas estas operaciones y haces todo en un solo bucle, la aceleración de algunos objetos se calculará a partir de $\mathbf{x}_i(n + 1)$ y no de $\mathbf{x}_i(n)$.

Truco: Para evitar otra asignación de memoria puedes añadir un campo **acceleration** a **OrbitalBody**, que utilizas en **updateOrbitalSim** como variable temporal entre el primer y segundo paso.

Para liberar los recursos de una simulación, completa la función **freeOrbitalSim**.

Consejos:

- **Aprovecha la biblioteca de vectores raymath que viene con raylib:** Tan solo tienes que escribir `#include "raymath.h"`. Los siguientes tipos y funciones son particularmente útiles: [Vector3](#), [Vector3Add](#), [Vector3Subtract](#), [Vector3Length](#) y [Vector3Scale](#).
- **Optimiza el cálculo:** Para calcular x^2 , $(x * x)$ requiere muchísimo menos cálculo que `pow(x, 2)`. También puedes ahorrar cálculo evitando multiplicaciones y divisiones inútiles, como la multiplicación y división por m_i en $\mathbf{F}_{i,j}$ y $\mathbf{a}_i$, respectivamente.
- **Verifica condiciones límite:** Asegúrate que tu código no produce divisiones por cero u otras condiciones que puedan romper la simulación.
- **Usa el debugger:** Aprovecha nuestra guía de **debugging**.
- **Haz la prueba:** Para verificar el funcionamiento de tu simulación puedes usar la prueba que incluimos en `CMakeLists.txt`. Modifícala para que se acomode a tu **OrbitalSim**. Para más información consulta la guía de **CMake y CTest**.

Vista

Para representar los cuerpos celestes en 3D, completa la función `renderOrbitalSim3D` del archivo `orbitalSimView.cpp`.

Puedes usar la función `DrawSphere` de raylib para representar los cuerpos con su color. Para que los cuerpos quepan en la pantalla debes escalar los valores de posición de `OrbitalSim`; recomendamos usar el factor de escala **1E-11**. Para que las esferas sean visibles también te sugerimos escalar sus radios; la ecuación empírica $0.005F * \log f(\text{radius})$ da muy buenos resultados.

Si te alejas mucho, verás que las esferas dejan de ser visibles. Para evitar esto píntalas también con `DrawPoint3D`.

Para mostrar información adicional sobre la simulación, completa la función `renderOrbitalSim2D`. Es particularmente útil que llames a `DrawFPS` para mostrar la cantidad de fotogramas por segundo. También puedes mostrar el tiempo de simulación con `DrawText`. Para convertir el tiempo en una fecha en formato ISO puedes usar la función `getISODate` que te proveemos.

Trucos:

- **Usa el mouse:** Para moverte en la simulación, aprieta el botón izquierdo del mouse para hacer paneo, la tecla **ALT** junto al botón izquierdo del mouse para rotar, y la rueda del mouse para hacer zoom.
- **Aprieta F12:** Para hacer bonitas capturas de pantalla.

Pasos siguientes

¡Felicitaciones, hiciste tu propio Universe Sandbox! Pero aún quedan cosas por hacer:

1. Es probable que hayas usado `float` para representar las masas, distancias, velocidades y aceleraciones de la simulación. Verifica que el tipo de datos que elegiste tiene la precisión y el rango de valores necesarios. Escribe tu justificación en el encabezado de `orbitalSim.cpp`.
2. Modifica el código en `makeOrbitalSim` para añadir 500 asteroides. Puedes usar la función `placeAsteroid` que te proveemos. Asegúrate de completarla.
3. Habrás notado que la simulación del punto anterior se volvió terriblemente lenta. ¿Cuál es la complejidad algorítmica de la simulación? ¿Por qué? Escribe tu respuesta en el encabezado de `orbitalSim.cpp`.

4. Soluciona el problema anterior. Cuando hayas solucionado el problema, deberías poder simular sin problema 1000 asteroides o más. Pista: hay dos cuellos de botella: uno que involucra a los gráficos, y otro, a la complejidad. Describe los cambios que hiciste en el encabezado de **orbitalSim.cpp**.
5. Añade pruebas si lo consideras necesario. Justifícalas con comentarios.

Bonus points

Más cosas (optativas) para hacer:

- Prueba qué pasaría si Jupiter fuera 1000 veces más masivo, o si un agujero negro pasara por en medio del sistema solar. Aprecia el hecho que estés con vida.
- En `ephemerides.h` hay unas efemérides para el sistema Alpha Centauri, la segunda y tercera estrella más cercana al sistema solar. ¡Pruébalas en tu simulador!
- Implementa tu propio [Kerbal Space Program](#) con un simulador de nave espacial.
- Encuentra el easter egg.